

Python Topic 11 — Error Handling (try / except)

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert

Layer 1 — New to Python

Sometimes code fails — a file's missing, a number won't convert, an API is down. Instead of crashing, you can **catch** the error and handle it gracefully.

```
try:
    number = int("hello")    # this fails - "hello" isn't a number
except:
    print("That wasn't a number!") # runs instead of crashing
```

try = "attempt this risky code." **except** = "if it breaks, do this instead." Your program keeps running.

```
try:
    result = 10 / 0          # division by zero - an error
except:
    print("Can't divide by zero")
```

Layer 2 — Intermediate (real code + how it works)

Catch specific errors (best practice)

```
try:
    age = int(input("Age: "))
except ValueError:
    print("Please enter a valid number")
```

JS comparison

```
// JS
try {
    risky();
} catch (error) {
    console.log(error.message);
}
```

```
# Python – name the exception type
try:
    risky()
except Exception as e:
    print(e)
```

Python names the *type* of error (`ValueError` , `KeyError` , etc.), which is more precise than JS's single `catch` .

Multiple except blocks

```
try:
    data = json.loads(text)
    value = data["key"]
    result = 100 / value
except json.JSONDecodeError:
    print("Invalid JSON")
except KeyError:
    print("Missing 'key'")
except ZeroDivisionError:
    print("Value was zero")
```

Capture the error object with `as`

```
try:
    int("abc")
except ValueError as e:
    print(f"Error: {e}")    # Error: invalid literal for int() with base 10: 'abc'
```

`else` and `finally`

```
try:
    file = open("data.txt")
except FileNotFoundError:
    print("File missing")
else:
    print("Success!")      # runs ONLY if no exception
    data = file.read()
finally:
    print("Always runs")   # cleanup – runs no matter what
```

- `else` → runs if the `try` succeeded (no exception)
- `finally` → always runs, error or not (cleanup: closing files/connections)

Common built-in exceptions

Exception	When it happens
<code>ValueError</code>	wrong value (<code>int("abc")</code>)
<code>TypeError</code>	wrong type (<code>"2" + 2</code>)
<code>KeyError</code>	missing dict key
<code>IndexError</code>	list index out of range
<code>FileNotFoundError</code>	file doesn't exist
<code>ZeroDivisionError</code>	divide by zero
<code>AttributeError</code>	object has no such attribute/method

Layer 3 — Expert (internals & gotchas)

1. Never use a bare `except:`

```
# ❌ BAD – catches EVERYTHING, even Ctrl+C and system exits
try:
    risky()
except:
    pass

# ✅ GOOD – catch what you expect
try:
    risky()
except (ValueError, KeyError) as e:
    logger.error(f"Failed: {e}")
```

 **Why:** a bare `except:` even catches `KeyboardInterrupt` and `SystemExit`, making your program impossible to stop. Always name the exception(s).

2. Raising your own exceptions

```
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("Insufficient funds")
    return balance - amount

# custom exception class
```

```
class InsufficientFundsError(Exception):
    pass

raise InsufficientFundsError("Not enough money")
```

`raise` throws an exception. Custom classes (inheriting from `Exception`) make your errors meaningful and catchable by type — you'll define these in your APIs.

3. Exception chaining with `raise ... from`

```
try:
    config = json.loads(raw)
except json.JSONDecodeError as e:
    raise ConfigError("Bad config file") from e  # preserves original cause
```

Keeps the original traceback while raising a higher-level error — great for clean error reporting in production.

4. The `finally` gotcha — runs even after `return`

```
def func():
    try:
        return "from try"
    finally:
        print("finally runs")  # prints BEFORE the return completes!

func()  # prints "finally runs", then returns "from try"
```

⚠ **Note:** `finally` runs no matter how you leave the `try` — even `return`, `break`, or another exception. Don't `return` inside `finally` (it silently swallows exceptions).

5. Prefer context managers (`with`) over try/finally for resources

```
# ❌ manual cleanup
file = open("data.txt")
try:
    data = file.read()
finally:
    file.close()

# ✅ context manager – auto-closes, even on error
with open("data.txt") as file:
    data = file.read()

# file is automatically closed here, exception or not
```

✓ **Pythonic:** `with` handles files, DB connections, locks — anything needing cleanup. It calls the cleanup for you. You'll use this constantly.

6. EAFP vs LBYL — Python's philosophy

```
# LBYL (Look Before You Leap) – check first
if "key" in data:
    value = data["key"]

# EAFP (Easier to Ask Forgiveness than Permission) – just try
try:
    value = data["key"]
except KeyError:
    value = default
```

Python **prefers EAFP** — attempt the operation and handle failure, rather than checking preconditions. Often faster and avoids race conditions (the key could vanish between check and access).

7. Don't use exceptions for normal control flow (with nuance)

```
# ✗ using exceptions where a check is clearer
try:
    return mylist[0]
except IndexError:
    return None

# ✓ often clearer
return mylist[0] if mylist else None
```

Exceptions are for *exceptional* situations. But EAFP is idiomatic when failure is rare — balance readability.

Interview Q&A Cards

Q: Why avoid a bare `except:` ?

A: It catches everything — including `KeyboardInterrupt` and `SystemExit` — hiding bugs and making the program impossible to interrupt. Always name specific exception types.

Q: `else` vs `finally` in a try block?

A: `else` runs only if the `try` succeeded with no exception. `finally` always runs (success, exception, or return) — used for cleanup.

Q: What's EAFP and does Python prefer it?

A: "Easier to Ask Forgiveness than Permission" — try the operation and catch failure, rather than checking first (LBYL). Python prefers EAFP; it's often faster and avoids race conditions.

Q: Why use `with` instead of try/finally?

A: A context manager (`with`) guarantees cleanup (closing files, connections, locks) even on exceptions, with less boilerplate than manual try/finally. It's the idiomatic approach.

🔗 **Memory hook:** `try` risky, `except SpecificError` handle, `else` if it worked, `finally` always.
Never bare `except` . Use `with` for cleanup. Ask forgiveness, not permission.

✅ **One-liner for interviews:** "Python uses `try / except / else / finally` . Always catch specific exception types, never bare `except` . `raise` throws; custom exceptions inherit from `Exception` . `finally` always runs (even after return); prefer `with` context managers for cleanup. Python favors EAFP — try and handle — over LBYL checking."